Wizardry — second wave: prove what is already built

Written 12 November 2026, the same day as the first plan. Measured on the live instance; every number names the command that produced it.

The first wave protects the instance: backups, an invite door, a gate, an operator CLI, some art. This wave is a different kind of work. **Fluxer already ships almost everything a member asks for.** Search, web push, data export, voice, invites, an Electron desktop shell — all of it is in the tree. What is missing is not code. What is missing is **proof**, and in one case a single fact that turns out to be broken.

So the second wave is not a feature list. It is a list of measurements, each with the small piece of glue that makes the measurement possible.

1. Facts

Fact	Value	How known	
Messages	9 — it was 8 when the first plan was written	`count(*)` on `messages`	
Message search index	**0 documents**; index created 14:01 today and never updated since	`GET /indexes/messages`	
User search index	2 documents	`GET /indexes/users/stats`	
Search indexes present	`audit_logs`, `guild_members`, `guilds`, `users`, `messages`	`GET /indexes`	
Push	relay consent accepted 14:20 by `tibs`; **0 devices subscribed**	push config; no subscription rows	
Web push in the tree	11 server files	`grep -ril push_subscription\`	webpush fluxer_api/src`
Voice	1 region, 1 server, **0 calls ever placed**	`voice_regions`, `voice_servers`	
Administrators	**1** account with `acls: ["*"]`; the other holds none	`users` rows	
Attachments	5	`attachment*` tables	
Data export in the tree	9 files reference it; **no export has ever been produced**	`grep -ril exportData`; no artifact on disk	
Desktop releases	**0**	`api.github.com/repos/.../releases`	
Read states	5 — channels have been opened	`read_states`	

Fact	Value	How known
Admin actions logged	186	`admin_audit_logs`

Two of those deserve their own sentence. The **message index holds zero documents** while the user index holds two: search is switched on, wired to Meilisearch, and indexing nothing you would ever want to find. And there is **one administrator**, so if that one account loses its password the admin panel has no second key.

2. Questions, and why the answer is not "it works"

#	Question	What exists	Why it is unproven	Kind
1	"What if I lose my admin account?"	one account with `acls: ["*"]`	nothing else can administer the instance	finding
2	"Can I find that message?"	Meilisearch, five indexes, a search UI	message index has 0 documents	finding
3	"Will I know when someone messages me?"	web push, relay consent given	no device has ever subscribed	finding
4	"Can I take my data out?"	export code in nine files	no export has ever been produced	finding
5	"Does a call actually work?"	LiveKit, one region, doctor green	zero calls placed	finding
6	"Can I run it as a desktop app?"	rebranded Electron sources in the fork	zero releases; no artifact ever built	finding

3. The six measurements

3.1 A second key to the admin panel

First, because everything else depends on being able to administer.

Give a second account administrative access, or a keeper account that exists only for recovery, and write down the path back in if the primary is lost.

- Rules: the second admin is a named person, not a shared login; the recovery path is executed once as a drill rather than described; the drill does not depend on a session that only the lost account could create.
- Pass: a second account opens `/admin` and sees the overview; the recovery drill is run once end to end with the time it took recorded; `users` shows exactly two rows with `acls: ["*"]` and no more.

3.2 Search that returns the message

Second, because it is broken rather than merely unproven.

Fill the message index, then prove a query finds a word that exists and returns nothing for one that does not.

- Rules: the index is filled by the instance's own indexing path, never by inserting documents by hand; a rebuild is a supported operation, not a one-off script; an empty result reads as "nothing found", not as an error.
- Pass: `numberOfDocuments` on the `messages` index equals the `messages` row count; searching a distinctive word from message 3 returns message 3 and only message 3; searching nonsense returns the empty state; both checked at 390 px and in the desktop client.

3.3 A notification that reaches a phone

Third, because "push is configured" and "my phone buzzed" are different claims.

- Rules: it must arrive with the tab closed, not only while a window is focused; the test message comes from the second account, not from the same user; a failed subscription is reported in words a person can act on.
- Pass: one device subscribed; a message from the other account raises a notification within 10 seconds with the browser closed; the notification opens the right channel; the subscription survives a browser restart.

3.4 An export that produces a file

Fourth, because "you can take your data" is a promise until a file exists.

- Rules: the export is the upstream path, untouched; what it contains is stated before it is trusted; a missing attachment is a failure, not a partial success.
- Pass: an export downloads and unzips; it contains all 9 messages and all 5 attachments; the file's size and timestamp are recorded; a second import of it is refused or explained rather than silently half-applied.

3.5 A call that has actually been made

Fifth, because a green health check is not a working microphone.

- Rules: a real two-device call, not a self-join; measured join time and measured audio in both directions; the result is recorded whether it passes or fails.
- Pass: 60 seconds of two-way audio; join under 3 seconds on the same network; the call ends cleanly with no orphaned voice state left in the store.

3.6 A desktop build that exists — or a decision that it never will

Sixth, because a rebranded shell nobody can download is a claim, not a product.

- Rules: either produce one artifact for one platform and launch it, or strike the desktop claim from `REBRAND.md` and say plainly that the web app is the client. Both are honest; leaving it ambiguous is not.
- Pass (if built): an artifact in GitHub releases, launched, showing the seal and the name in the title bar and about box, with the version matching the server.
- Pass (if struck): `REBRAND.md` lists the desktop identity files as unused, and no document claims a desktop build.

4. Order, and across waves

1. **3.1 second admin** — the only item that protects the ability to fix the rest.

2. **3.2 search** — already broken; cheapest genuine repair on the list.
3. **3.3 push** — needs a phone and a second account, both of which exist.
4. **3.4 export** — one run, one file, one record.
5. **3.5 call** — needs two devices and a quiet ten minutes.
6. **3.6 desktop** — a build or a deletion; both are cheap and neither is urgent.

Across waves: wave 1 runs first except for 3.1 and 3.2, which are cheap and belong before the gate is built — there is no sense proving an instance that cannot be administered or searched. Everything else in this wave waits until wave 1's restore drill exists, because every measurement below writes data.

5. Considered and rejected

- **New messaging features.** Upstream has threads, forums, reactions, invites with expiry, DMs, read states. Nine messages do not need a tenth feature.
- **Federation / ActivityPub.** There is signing material in the tree and no audience for it. A private instance that federates stops being private.
- **A mobile app.** Fluxer's mobile client is a separate Flutter repository; forking a second stack to serve two accounts is a maintenance bill with no invoice.
- **Screenshare and stream tuning.** Rejected until a call has been made at all — 3.5 is the prerequisite, not a competitor.
- **Theme v2, badges, bots, an agent in the guild.** Same reasons as wave 1: one accent, no audience, and authorship stays human.
- **Rewriting the search indexing path to be "better".** The upstream path is the one the project tests; the fix is to make it run here, not to out-design it.

6. Correction

Wave 1 recorded **8 messages**; this wave records **9**. The number moved while the planning was happening, which is the most useful fact in either document: the instance is not a museum piece, someone is using it, and any plan that treats it as frozen will be wrong within the hour. Both documents keep the number they measured, with the date, rather than being quietly updated to agree.

7. Open questions for the owner

1. Who is the second administrator — `eleven`, or a keeper account that never posts?
2. Is this week realistic for the phone test in 3.3, or should it wait for a device you actually carry?
3. Desktop: build one for Linux (cheap, testable here) or strike the claim?

8. When to stop planning

This is the second wave, and it is the last one worth writing. Beyond it the honest move is to stop producing documents and start producing artifacts, in this order: the restore drill, the invite door, the second admin, the message index. If a third document is ever tempting, the test is whether the first two waves have any ticked boxes; if they do not, a third plan is a way of avoiding the first one.

Change log

- 12 Nov 2026 — first issue. Written after wave 1, measured an hour later: 9 messages, 0 indexed, 1 admin, 0 releases.